

Exercice 1 :

Les participants à un jeu de LaserGame sont répartis en équipes et s'affrontent dans ce jeu de tir, revêtus d'une veste à capteurs et munis d'une arme factice émettant des infrarouges. Les ordinateurs embarqués dans ces vestes utilisent la programmation orientée objet pour modéliser les joueurs. La classe Joueur est définie comme suit :

```
class Joueur:
2     def __init__(self, pseudo, identifiant, equipe):
3         """ constructeur """
4         self.pseudo = pseudo
5         self.equipe = equipe
6         self.id = identifiant
7         self.nb_de_tirs_emis = 0
8         self.liste_id_tirs_recus = []
9         self.est_actif = True
10
11     def tire(self):
12         """méthode déclenchée par l'appui sur la gachette"""
13         if self.est_actif == True:
14             self.nb_de_tirs_emis = self.nb_de_tirs_emis + 1
15
16     def est_determine(self):
17         """methode qui renvoie True si le joueur réalise
18         un grand nombre de tirs"""
19         return self.nb_de_tirs_emis > 500
20
21     def subit_un_tir(self, id_recu):
22         """méthode déclenchée par les capteurs de la
23         veste"""
24         if self.est_actif == True:
25             self.est_actif = False
26             self.liste_id_tirs_recus.append(id_recu)
```

1. Parmi les instructions suivantes, recopier celle qui permet de déclarer un objet joueur1, instance de la classe Joueur, correspondant à un joueur dont le pseudo est "Sniper", dont l'identifiant est 319 et qui est intégré à l'équipe "A":

Instruction 1 : joueur1 = ["Sniper", 319, "A"]

Instruction 2 : joueur1 = new Joueur["Sniper", 319, "A"]

Instruction 3 : joueur1 = Joueur("Sniper", 319, "A")

Instruction 4 : joueur1 = Joueur{"pseudo":"Sniper",
"id":319, "equipe":"A"}

2. La méthode subit_un_tir réalise les actions suivantes :

Lorsqu'un joueur actif subit un tir capté par sa veste, l'identifiant du tireur est ajouté à l'attribut liste_id_tirs_recus et l'attribut est_actif prend la valeur False (le joueur est désactivé). Il doit alors revenir à son camp de base pour être de nouveau actif.

- a. Écrire la méthode `redevenir_actif` qui rend à nouveau le joueur actif uniquement s'il était précédemment désactivé.
 - b. Écrire la méthode `nb_de_tirs_recus` qui renvoie le nombre de tirs reçus par un joueur en utilisant son attribut `liste_id_tirs_recus`.
3. Lorsque la partie est terminée, les participants rejoignent leur camp de base respectif où un ordinateur, qui utilise la classe `Base`, récupère les données.

La classe `Base` est définie par :

- ses attributs :

- `equipe` : nom de l'équipe (str). Par exemple, "A" ,
- `liste_des_id_de_l_equipe` qui correspond à la liste (list) des identifiants connus des joueurs de l'équipe,
- `score` : score (int) de l'équipe, dont la valeur initiale est 1000 ;

- ses méthodes :

- `est_un_id_allie` qui renvoie `True` si l'identifiant passé en paramètre est un identifiant d'un joueur de l'équipe, `False` sinon,
- `incremente_score` qui fait varier l'attribut `score` du nombre passé en paramètre,
- `collecte_information` qui récupère les statistiques d'un participant passé en paramètre (instance de la classe `Joueur`) pour calculer le score de l'équipe .

```

1     def collecte_information(self, participant):
2         if participant.equipe == self.equipe : # test 1
3             for id in participant.liste_id_tirs_recus:
4                 if self.est_un_id_allie(id): # test 2
5                     self.incremente_score(-20)
6                 else:
7                     self.incremente_score(-10)

```

- a. Indiquer le numéro du test (test 1 ou test 2) qui permet de vérifier qu'en fin de partie un participant égaré n'a pas rejoint par erreur la base adverse. **Test 1**
- b. Décrire comment varie quantitativement le score de la base lorsqu'un joueur de cette équipe a été touché par le tir d'un coéquipier. **-20**

On souhaite accorder à la base un bonus de 40 points pour chaque joueur particulièrement déterminé (qui réalise un grand nombre de tirs).

Recopier et compléter, en utilisant les méthodes des classes `Joueur` et `Base`, les 2 lignes de codes suivantes qu'il faut ajouter à la fin de la méthode `collecte_information` :

```

..... #si le participant réalise un grand nombre de tirs
..... #le score de la Base augmente de 40

```

Exercice 2 :

Simon souhaite créer en Python le jeu de cartes « la bataille » pour deux joueurs.

Les questions qui suivent demandent de reprogrammer quelques fonctions du jeu. On rappelle ici les règles du jeu de la bataille :

Préparation :

- Distribuer toutes les cartes aux deux joueurs.
- Les joueurs ne prennent pas connaissance de leurs cartes et les laissent en tas face cachée devant eux.

Déroulement :

- A chaque tour, chaque joueur dévoile la carte du haut de son tas.
- Le joueur qui présente la carte ayant la plus haute valeur emporte les deux cartes qu'il place sous son tas.
- **Les valeurs des cartes sont** : dans l'ordre de la plus forte à la plus faible : As, Roi, Dame, Valet, 10, 9, 8, 7, 6, 5, 4, 3 et 2 (la plus faible)

Si deux cartes sont de même valeur, il y a "bataille".

- Chaque joueur pose alors une carte face cachée, suivie d'une carte face visible sur la carte dévoilée précédemment.
- On recommence l'opération s'il y a de nouveau une bataille sinon, le joueur ayant la valeur la plus forte emporte tout le tas.

Lorsque l'un des joueurs **possède toutes les cartes du jeu**, la partie s'arrête et ce dernier gagne.

Pour cela Simon crée une classe Python `Carte`. Chaque instance de la classe a deux attributs : un pour sa valeur et un pour sa couleur. Il donne au valet la valeur 11, à la dame la valeur 12, au roi la valeur 13 et à l'as la valeur 14. La couleur est une chaîne de caractères : "trefle", "carreau", "coeur" ou "pique".

1. Simon a écrit la classe Python `Carte` suivante, ayant deux attributs `valeur` et `couleur`, et dont le constructeur prend deux arguments : `val` et `coul`.

- a. Recopier et compléter les des lignes 3 et 4 ci-dessous.

```
class Carte:
2.     def __init__(self, val, coul):
3.         .....self.valeur = .....val
4.         .....self.couleur = coul
```

- b. Parmi les propositions ci-dessous quelle instruction permet de créer l'objet « 7 de cœur » sous le nom `c7` ?

- `c7.__init__(self, 7, "coeur")`
- `c7 = Carte(self, 7, "coeur")`
- `c7 = Carte(7, "coeur")`
- `from Carte import 7, "coeur"`

2. On souhaite créer le jeu de cartes. Pour cela, on écrit une fonction `initialiser()` :

- sans paramètre
- qui renvoie une liste de 52 objets de la classe `Carte` représentant les 52 cartes du jeu.

Voici une proposition de code. Recopier et compléter les lignes suivantes pour que la fonction réponde à la demande :

```
def initialiser() :  
    jeu = []  
    for c in ["coeur", "carreau", "trefle", "pique"] : # couleur  
        carte  
            for v in range(2,15) : # valeur carte  
                carte_cree = Carte(v,c)  
                jeu.append(carte_cree)  
    return jeu
```

3. On rappelle que dans une partie de bataille, les deux joueurs tirent chacun une carte du dessus de leur tas, et celui qui tire la carte la plus forte remporte les deux cartes et les place en dessous de son tas.

Parmi les structures linéaires de données suivantes : Tableau, File, Pile, quelle est celle qui modélise le mieux un tas de cartes dans ce jeu de la bataille ? Justifier votre choix.

File ?

4. Écrire une fonction `comparer(cartel, carte2)` qui prend en paramètres deux objets de la classe `Carte`. Cette fonction renvoie :

- 0 si la force des deux cartes est identique,
- 1 si la carte `cartel` est strictement plus forte que `carte2`
- -1 si la carte `carte2` est strictement plus forte que `cartel`

Exercice3 :

Un fabricant de brioches décide d'informatiser sa gestion des stocks. Il écrit pour cela un programme en langage Python. Une partie de son travail consiste à développer une classe `Stock` dont la première version est la suivante :

```
class Stock:  
  
    def __init__(self):  
  
        self.qt_farine = 0 # quantité de farine initialisée à 0 g  
        self.nb_oeufs = 0 # nombre d'œufs (0 à l'initialisation)  
        self.qt_beurre = 0 # quantité de beurre initialisée à 0 g
```

1. Écrire une méthode `ajouter_beurre(self, qt)` qui ajoute la quantité `qt` de beurre à un objet de la classe `Stock`.

On admet que l'on a écrit deux autres méthodes `ajouter_farine` et `ajouter_oeufs` qui ont des fonctionnements analogues.

2. Écrire une méthode `afficher(self)` qui affiche la quantité de farine, d'œufs et de beurre d'un objet de type `Stock`. L'exemple ci-dessous illustre l'exécution de cette méthode dans la console :

```
>>> mon_stock = Stock()
>>> mon_stock.afficher()
farine: 0
oeuf: 0
beurre: 0
>>> mon_stock.ajouter_beurre(560)
>>> mon_stock.afficher()
farine: 0
oeuf: 0
beurre: 560
```

3. Pour faire une brioche, il faut 350 g de farine, 175 g de beurre et 4 œufs. Écrire une méthode `stock_suffisant_brioche(self)` qui renvoie un booléen : VRAI s'il y a assez d'ingrédients dans le stock pour faire une brioche et FAUX sinon.

4. On considère la méthode supplémentaire `produire(self)` de la classe `Stock` donnée par le code suivant :

```
def produire(self):
```

```
    res = 0

    while self.stock_suffisant_brioche():
        self.qt_beurre = self.qt_beurre - 175
        self.qt_farine = self.qt_farine - 350
        self.nb_oeufs = self.nb_oeufs - 4
        res = res + 1

    return res
```

On considère un stock défini par les instructions suivantes :

```
>>> mon_stock=Stock()
>>> mon_stock.ajouter_beurre(1000)
>>> mon_stock.ajouter_farine(1000)
>>> mon_stock.ajouter_oeufs(10)
```

- a. On exécute ensuite l'instruction

```
>>> mon_stock.produire() 2 soit deux briches
```

Quelle valeur s'affiche dans la console ? Que représente cette valeur ?

- b. On exécute ensuite l'instruction

```
>>> mon_stock.afficher() farine: 300
oeuf: 2
beurre: 650
```

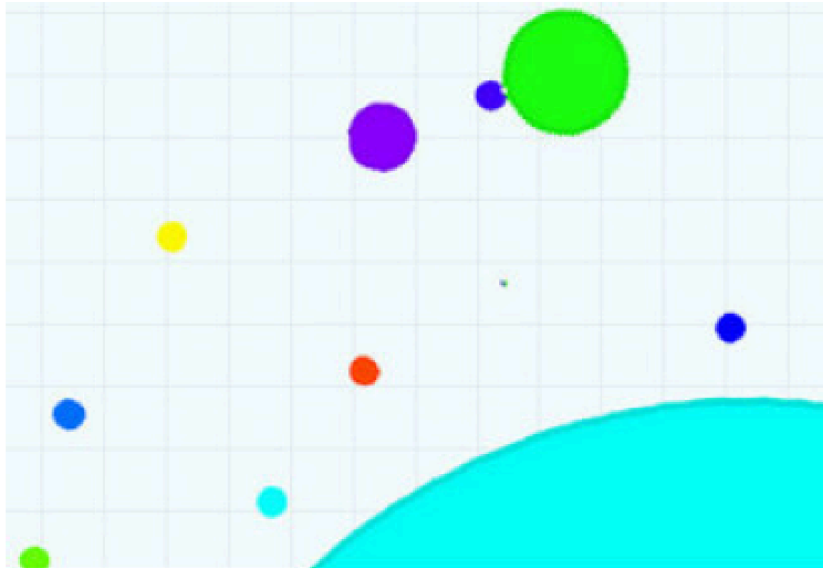
Que s'affiche-t-il dans la console ?

5. L'industriel possède n lieux de production distincts et donc n stocks distincts. On suppose que ces stocks sont dans une liste dont chaque élément est un objet de type `Stock`.

Écrire une fonction Python `nb_brioche(liste_stocks)` possédant pour unique paramètre la liste des stocks et renvoie le nombre total de brioches produites.

Exercice 4 :

Dans un jeu de plateforme, des bulles de couleurs et de diamètres différents se déplacent de manière aléatoire. A chaque fois qu'une bulle touche une bulle plus grande, la petite cède son contenu à la plus grande, et donc celle-ci augmente de surface. Par exemple, si une bulle de 1 cm^2 rencontre une bulle de 4 cm^2 , la petite bulle disparaît et la plus grande a désormais une surface de 5 cm^2 . A chaque collision, la vitesse de la grande bulle est réduite de moitié.



Le développeur a choisi de coder en Python, chaque bulle est un objet disposant entre autre des attributs suivants :

- `xc, yc` sont deux entiers, les coordonnées du pixel placé au centre de la bulle,
- `rayon` est un entier, le rayon de la bulle en pixels,
- `couleur` est un entier, la couleur de la bulle,
- `dirx, diry` sont deux décimaux (float) qui déterminent les déplacements à l'horizontale et à la verticale à chaque fois que la bulle se déplace. Ces deux valeurs déterminent donc la direction et la vitesse de la bulle. Par exemple si `dirx` vaut `0.5` et `diry` vaut `0.0`, la bulle se déplace vers la droite uniquement alors que si `dirx` vaut `-1.0` et `diry` vaut `0.0`, la bulle se déplace vers la gauche et deux fois plus vite que précédemment.

On suppose que toutes les fonctions de la bibliothèque `math` ont déjà été importées par l'instruction `from math import *`.

La fonction `randint` de la bibliothèque `random` prend en paramètre deux entiers et renvoie un entier aléatoire dans la plage définie par les deux paramètres.

Exemple : randint (-1, 5) peut renvoyer une des valeurs suivantes : -1, 0, 1, 2, 3, 4, 5.

1. Pour simplifier, on se limitera à un jeu de six bulles. Au départ, on crée une liste appelée Mousse de longueur six contenant six emplacements vides : `Mousse = [None, None, None, None, None, None]`

Le code ci-dessous montre le début du programme et notamment la structure définition de la classe nommée Cbulle ainsi que le code permettant le déplacement d'une bulle.

```
from random import randint
from math import *
class Cbulle:
    def __init__(self):
        self.xc = randint(0, 100)
        self.yc = randint(0, 100)
        self.rayon = randint(0, 10)
        self.dirx = float(randint(-1, 1)) # dirx et diry valent
        self.diry = float(randint(-1, 1)) # -1.0 ou 0.0. ou 1.0
        self.couleur = randint(1, 65535)
    def bouge(self):
        # déplace la bulle
        self.xc = self.xc + self.dirx
        self.yc = self.yc + self.diry
```

On crée les six bulles une à une et ces objets sont stockés dans les emplacements vides de la liste Mousse.

```
Mousse = [bulle1, bulle2, bulle3, bulle4, bulle5, bulle6]
```

Lors d'une collision, la bulle la plus petite disparaît et est remplacée dans la liste par la valeur None tandis que la plus grosse a sa surface qui augmente.

Au cours d'une partie, si une ou plusieurs bulles ont disparu, le programme peut en introduire de nouvelles dans le jeu: dans ce cas, lorsqu'une nouvelle bulle apparaît, elle remplace le premier None de la liste Mousse.

- a. Recopier les quatre dernières lignes et compléter les du code python ci-dessous.

```
def donnePremierIndiceLibre(Mousse):
    """
    Mousse est une liste.
    La fonction doit renvoyer l'indice du premier
    emplacement libre (contenant None) dans la liste Mousse
    ou renvoyer 6 en l'absence d'un emplacement libre dans
    Mousse."""
    i = 0
    while i <= len(Mousse) and Mousse[i] != None :
        i += 1
    return i
```

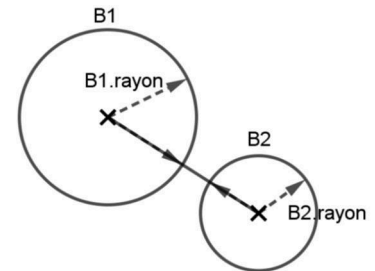
- b. Lorsque le jeu crée une bulle (instance de la classe Cbulle), il doit ensuite la placer dans la liste Mousse à la place d'un None.

Ecrire la fonction `placeBulle (B)` qui reçoit en paramètre un objet de type `Cbulle` et qui place cet objet dans la liste `Mousse`. Cette fonction ne renvoie rien, mais la liste `Mousse` est modifiée. Si aucun emplacement n'est disponible, la fonction ne modifie rien.

2. Pour le bon déroulement du jeu, on a besoin aussi d'une fonction `bullesEnContact (B1, B2)` qui renvoie `True` si la bulle `B2` touche la bulle `B1` et `False` dans le cas contraire.

On peut remarquer que deux bulles sont en contact si la distance qui sépare leur centre est inférieure ou égale à la somme de leurs rayons.

On dispose de la fonction `distanceEntreBulles (B1, B2)` qui calcule et renvoie la distance entre les centres de bulles `B1` et `B2`.



Écrire la fonction `bullesEnContact (B1, B2)`.

3. Quand une petite bulle touche une plus grosse bulle, on appelle la fonction `collision`, ci-dessous, où `indPetite` est l'indice de la petite bulle et `indGrosse` l'indice de la grosse bulle dans `Mousse`.

Recopier et compléter les de la fonction `collision`.

```
def collision(indPetite, indGrosse, mousse) :
    """
    Absorption de la plus petite bulle d'indice indPetite
    par la plus grosse bulle d'indice indGrosse. Aucun test
    n'est réalisé sur les positions.
    """
    # calcul du nouveau rayon de la grosse bulle
    surfPetite = pi*Mousse[indPetite].rayon**2
    surfGrosse = pi*Mousse[indGrosse].rayon**2
    surfGrosseApresCollision = surfGrosse + surfPetite
    rayonGrosseApresCollision =
sqrt(surfGrosseApresCollision/pi)
    #réduction de 50% de la vitesse de la grosse bulle
    Mousse[indGrosse].dirx = .....Mousse[indGrosse].dirx/2
    Mousse[indGrosse].diry = .....Mousse[indGrosse].diry/2
    #suppression de la petite bulle dans Mousse
    Mousse[indPetite] = None
    Mousse[indGrosse].rayon = rayonGrosseApresCollision # car l affecte pas
```

Exercice 5

Notion abordée : programmation objet.

Cryptage selon le « Code de César »

Dans cet exercice, on étudie une méthode de chiffrement de chaînes de caractères alphabétiques. Pour des raisons historiques, cette méthode de chiffrement est appelée "code de César". On considère que les messages ne contiennent que les lettres capitales de l'alphabet "ABCDEFGHIJKLMNOPQRSTUVWXYZ" et la méthode de chiffrement utilise un nombre entier fixé appelé la clé de chiffrement.

1. Soit la classe `CodeCesar` définie ci-dessous :

class `CodeCesar`:

```
def __init__(self, cle):
    self.cle = cle
```

```

        self.alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    def decale(self, lettre):
        num1 = self.alphabet.find(lettre)
        num2 = num1+self.cle
        if num2 >= 26:
            num2 = num2-26
        if num2 < 0:
            num2 = num2+26
        nouvelle_lettre = self.alphabet[num2]
        return nouvelle_lettre

```

On rappelle que la méthode `str.find(lettre)` renvoie l'indice (index) de la lettre dans la chaîne de caractères `str`. Représenter le résultat d'exécution du code Python suivant :

```

code1 = CodeCesar(3)
print(code1.decale('A'))  D
print(code1.decale('X'))  A

```

2. La méthode de chiffrement du « code César » consiste à décaler les lettres du message dans l'alphabet d'un nombre de rangs fixé par la clé. Par exemple, avec la clé 3, toutes les lettres sont décalées de 3 rangs vers la droite : le A devient le D, le B devient le E, etc.

Ajouter une méthode `cryptage(self, texte)` dans la classe `CodeCesar` définie à la question précédente, qui reçoit en paramètre une chaîne de caractères (le message à crypter) et qui retourne une chaîne de caractères (le message crypté). Cette méthode `cryptage(self, texte)` doit crypter la chaîne texte avec la clé de l'objet de la classe `CodeCesar` qui a été instancié.

Exemple :

```

>>> code1 = CodeCesar(3)
>>> code1.cryptage("NSI")
'QVL'

```

3. Ecrire un programme qui :

- Demande de saisir la clé de chiffrement
- Crée un objet de classe `CodeCesar`
- Demande de saisir le texte à chiffrer
- Affiche le texte chiffré en appelant la méthode `cryptage`

4. On ajoute la méthode `transforme(texte)` à la classe `CodeCesar` :

```

def transforme(self, texte):
    self.cle = -self.cle
    message = self.cryptage(texte)
    self.cle = -self.cle
    return message

```

On exécute la ligne suivante : `print(CodeCesar(10).transforme("PSX"))` **FIN ca affiche le cryptage avec la cle negatif**
 Que va-t-il s'afficher ? Expliquer votre réponse.

Exercice 6 :

Cet exercice porte sur la programmation orientée objet.

Une agence immobilière développe un programme pour gérer les biens immobiliers qu'elle propose à la vente.

Dans ce programme, pour modéliser les données de biens immobiliers, on définit une classe

`Bim` avec les attributs suivants :

- `nt` de type `str` représente la nature du bien (appartement, maison, bureau, commerces, ...);
- `sf` de type `float` est la surface du bien ;
- `pm` de type `float` est le prix moyen par m² du bien qui dépend de son emplacement.

La classe `Bim` possède une méthode `estim_prix` qui renvoie une estimation du prix du bien. Le code (incomplet) de la classe `Bim` est donné ci-dessous :

```
class Bim:
    def __init__(self, nature, surface, prix_moy):
        ...
    def estim_prix(self):
        return self.sf * self.pm
```

1. Recopier et compléter le code du constructeur de la classe `Bim`.

2. On exécute l'instruction suivante :

```
b1 = Bim('maison', 70.0, 2000.0)
```

Renvoie un float 140000.0

Que renvoie l'instruction `b1.estim_prix()` ? Préciser le type de la valeur renvoyée.

3. On souhaite affiner l'estimation du prix d'un bien en prenant en compte sa nature :

Pour un bien dont l'attribut `nt` est 'maison' la nouvelle estimation du prix est le produit de sa surface par le prix moyen par m² multiplié par 1,1 ;

- Pour un bien dont l'attribut `nt` est 'bureau' la nouvelle estimation du prix est le produit de sa surface par le prix moyen par m² multiplié par 0,8 ;
- Pour les biens d'autres natures, l'estimation du prix ne change pas.

Modifier le code de la méthode `estim_prix` afin de prendre en compte ce changement de calcul.

4. Écrire le code Python d'une fonction `nb_maison(lst)` qui prend en argument une liste Python de biens immobiliers de type `Bim` et qui renvoie le nombre d'objets de nature 'maison' contenus dans la liste `lst`.